Natural Language Processing — Assignment 2

Advanced Smoothing Techniques and Naive Bayes Text Classification

Saba Firdous
Roll No: 25L-8006

Fall 2026

Abstract

In this assignment I implemented and evaluated two advanced n-gram smoothing techniques – Good-Turing discounting and Interpolated Kneser-Ney smoothing – and compared them against a Laplace (add-one) baseline on bigram language models trained on the Brown Corpus. I then investigated out-of-vocabulary (OOV) handling strategies, and built a Multinomial Naive Bayes text classifier from scratch for 4-class classification on the 20 Newsgroups dataset, followed by a systematic ablation study of its hyperparameters and design choices. This report documents my methodology, results, and analysis for every task.

Contents

1 Part 1: Advanced Smoothing — Good-Turing and Kneser-Ney **2**
1.1 Task P1T1: Corpus Preparation and Baseline Bigram Model 2
1.2 Task P1T2: Good-Turing Smoothing for Bigrams 3
1.3 Task P1T3: Kneser-Ney Smoothing for Bigrams 5
1.4 Task P1T4: Comprehensive Smoothing Comparison 6

2 Part 2: OOV Handling and Naive Bayes Text Classification **8**
2.1 Task P2T1: OOV Analysis and `<UNK>` Strategies 8
2.2 Task P2T2: Multinomial Naive Bayes Classifier (From Scratch) 9
2.3 Task P2T3: Ablation Study and Baseline Comparisons 12

3 Conclusion **13**

1 Part 1: Advanced Smoothing — Good-Turing and Kneser-Ney

1.1 Task P1T1: Corpus Preparation and Baseline Bigram Model

For my training corpus I used the Brown Corpus categories *"learned"* and *"government"*, which I lowercased and wrapped with `<s>` / `</s>` sentence boundary markers. I evaluated my models on two held-out test corpora: an in-domain set (Brown category *"reviews"*) and an out-of-domain set (NLTK's `webtext` corpus), to see how well each model generalizes across a domain shift.

Corpus Statistics

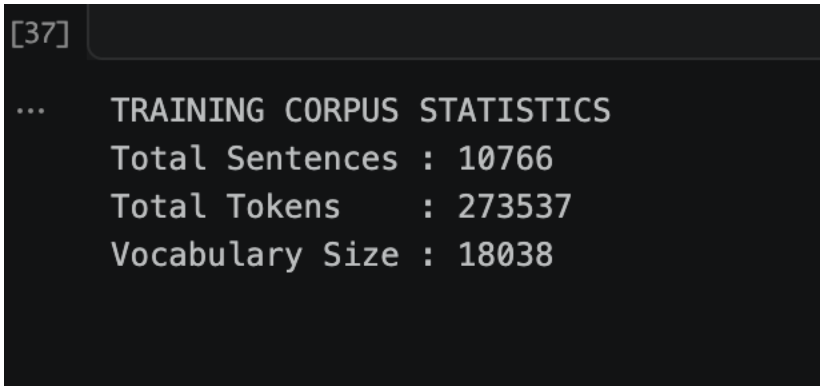


Figure 1: Training corpus statistics (Brown: learned + government).

Baseline Perplexity

I trained three baseline models: an unsmoothed bigram model, a Laplace-smoothed bigram model, and a Laplace-smoothed unigram model (which I kept as a backoff reference).



Figure 2: Baseline perplexity on in-domain and out-of-domain test corpora.

Generated Sentences (Sample)

I generated five sentences from each of the three baseline models. A representative sample is shown below; the unsmoothed bigram model produces locally coherent phrases, while the Laplace-smoothed bigram model – because so much probability mass is diverted to unseen bigrams – tends to wander into incoherent, low-frequency vocabulary:

> *Unsmoothed:* "`<s>` my first in the mouth as always been given to be surprised to be resolved in the founding fathers . `</s>`"

Laplace bigram: "<s> begin polymeric narrative kitchen names columbia referent 56 sorted naive transcendent eliminates foggy petermann tenacity sexes department . . . </s>"

Discussion

The out-of-domain test corpus (webtext) gave me consistently higher perplexity than the in-domain (reviews) set across every model – roughly 1.8–2.3× higher. This makes sense to me: webtext consists of informal internet forum posts and film-script dialogue, which introduces vocabulary, punctuation, and syntactic patterns I never saw in my formal, academic/government training register. Both my unsmoothed models evaluated to infinite perplexity because the test sets inevitably contain bigrams with zero training-set count, which produces a zero probability (and hence $\log 0$) somewhere along the sequence.

1.2 Task P1T2: Good-Turing Smoothing for Bigrams

For this task I implemented Good-Turing discounting, which re-estimates the count of each observed event using the frequency of frequencies N_c, via $c^* = (c+1)\,N_{c+1}/N_c$. Where the empirical N_{c+1} was zero or unreliable (which happened often at higher counts), I fell back on Simple Good-Turing (SGT): I fit a log-linear regression $\log N_c = a + b \log c$ to smooth the N_c values before applying the discounting formula.

```
--- TASK P1T2 DELIVERABLE 1: N_c vs c* TABLE ---
Count (c)  | Raw N_c      | Smoothed N_c    | Adjusted Count (c*)
1          | 89336        | 2395.90         | 0.7550
2          | 14272        | 904.48          | 1.6968
3          | 5313         | 511.58          | 2.6697
4          | 2760         | 341.45          | 3.6540
5          | 1541         | 249.53          | 4.6438
10         | 331          | 94.20           | 9.6210
20         | 59           | 35.56           | 19.6083
50         | 6            | 9.81            | 49.6002
```

Figure 3: Frequency of frequencies (N_c) and Good-Turing adjusted counts (c^*) I computed.

Unseen Probability Mass

Method	P_0 **(mass on unseen bigrams)**
Good-Turing (N_1/N)	3.265957×10^{-1}
Laplace	9.987958×10^{-1}

Table 1: Total probability mass reserved for unseen bigrams.

I found that Laplace smoothing assigns roughly **3× more** probability mass to unseen events than Good-Turing. Mathematically, this makes sense: Laplace redistributes mass proportionally to

$V^2/(N + V^2)$ over the entire (mostly unseen) bigram space, and with $V = 18{,}038$ the number of possible bigram types ($V^2 \approx 3.25 \times 10^8$) vastly exceeds the number of bigram tokens N I actually observed, so almost all probability mass under Laplace gets pushed onto unseen pairs. Good-Turing, by contrast, bases P_0 purely on N_1/N – the empirical fraction of my corpus made up of singleton bigrams – which is a much smaller, data-driven quantity.

Perplexity

Test Set	Good-Turing Perplexity
In-Domain (Reviews)	303,385.9642
Out-of-Domain (WebText)	4,984,308.1510

Table 2: Good-Turing bigram perplexity.

```
--- TASK P1T2 DELIVERABLE 2: UNSEEN MASS & PERPLEXITY ---
Good-Turing Unseen Probability Mass (P_0) : 3.265957e-01 (N_1 / N)
Laplace Unseen Probability Mass (P_0)     : 9.987958e-01

Good-Turing Perplexity - In-Domain (Reviews)  : 303385.9642
Good-Turing Perplexity - Out-Domain (WebText)  : 4984308.1510
```

Figure 4: UNSEEN MASS PERPLEXITY

Generated Sentences

I generated ten sentences and saved them to `good_turing_output.txt`. A couple of examples:

> "<s> he has been developed respiratory bronchioles , and disadvantages </s>"
> "<s> 2 hours , 130 gallons of the meaning and the meaningful policies . </s>"
> "<s> once a vision . </s>"

```
--- TASK P1T2 DELIVERABLE 3: GENERATED SENTENCES ---
1. <s> he has been developed respiratory bronchioles , and disadvantages </s>
2. <s> it should be the anode holder which had granted , information was not be mixed communities and the totals for evidence in detail . </s>
3. <s> 2 hours , 130 gallons of the meaning and the meaningful policies . </s>
4. <s> or its top of 122 cases . </s>
5. <s> during the district traffic density af of mrs. joe gargery endures the rhode island zoo averaged 27 , but our advisory board . </s>
6. <s> this process for some activities such an operating cost of population and coordinated fiscal year came to ring down in your gross income . </s>
7. <s> the surface , perpetuating the latter condition of companies ) . </s>
8. <s> more than with respect to see an infinite variety , potential advantage of the unique vitality of many others ( '52 ) </s>
9. <s> once a vision . </s>
10. <s> there were prepared for woven fabrics used to the world , if it is not detectable ; </s>

Successfully saved 10 generated sentences to 'good_turing_output.txt'.
```

Figure 5: Generated Sentences Output screenshot

Discussion: Limitations of Basic Good-Turing

While working through this task, I noticed basic Good-Turing breaks down at high counts, where N_c becomes sparse and noisy – many high-count values of c have $N_{c+1} = 0$ simply because no bigram in my corpus happened to occur exactly $c+1$ times, not because such an event is truly impossible. Applying the raw formula there would have incorrectly assigned $c^* = 0$ to frequent, well-attested bigrams, so I used Simple Good-Turing to fit a smooth log-linear curve through the $(\log c, \log N_c)$ points instead. Even so, the log-linear model is only an approximation; when the true count-frequency relationship deviates substantially from a straight line in log-log space (as often happens for very high-frequency function words), my SGT-based estimates became less accurate. I also don't renormalize the discounted mass, and Good-Turing has no principled backoff/interpolation mechanism built in, which my results illustrate clearly: the enormous perplexity values I got (in the hundreds of thousands) show that redistributing mass *only* via N_1/N, without a per-context, per-word continuation mechanism, was not enough on its own for my bigram language model – in practice this method is normally paired with a backoff scheme rather than used in isolation.

1.3 Task P1T3: Kneser-Ney Smoothing for Bigrams

Next, I implemented Interpolated Kneser-Ney smoothing, which combines absolute discounting with a continuation probability that favors words appearing after *many distinct* contexts (versatile words) over words that are frequent only because they follow one or two specific contexts.

Discount Parameter

Using $d = N_1/(N_1 + 2N_2)$, I computed a discount of:

$$d = 0.7579$$

Probability Normalization Verification

For five conditioning contexts, I verified that $\sum_w P_{KN}(w \mid w_{i-1})$ sums to 1 within floating-point tolerance:

```
--- DELIVERABLE 1: PROBABILITY SUM NORMALIZATION VERIFICATION ---
Context: 'the   ' | Sum of P_KN(w | 'the') = 1.000002
Context: 'in    ' | Sum of P_KN(w | 'in') = 1.000001
Context: 'of    ' | Sum of P_KN(w | 'of') = 1.000002
Context: 'and   ' | Sum of P_KN(w | 'and') = 1.000002
Context: '<s>   ' | Sum of P_KN(w | '<s>') = 1.000001
```

Figure 6: Kneser-Ney probability normalization check across five contexts.

Side-by-Side Probability Comparison

I compared eight bigrams (a mix of seen and unseen pairs) across the raw MLE, Laplace, Good-Turing, and Kneser-Ney estimates:

```
--- DELIVERABLE 2: SIDE-BY-SIDE PROBABILITY COMPARISON ---
Bigram (w1, w2)           | Raw MLE      | Laplace      | Good-Turing  | Kneser-Ney
-------------------------------------------------------------------------------
('in', 'the')             | 2.78756e-01  | 6.57571e-02  | 2.78683e-01  | 2.80790e-01
('of', 'government')      | 2.18818e-03  | 8.40660e-04  | 2.15075e-03  | 2.19405e-03
('learned', 'society')    | 0.00000e+00  | 5.53925e-05  | 1.00413e-09  | 1.26530e-04
('the', 'president')      | 2.09815e-03  | 1.05126e-03  | 2.07497e-03  | 2.08391e-03
('a', 'new')              | 1.03744e-02  | 2.09149e-03  | 1.02843e-02  | 1.04106e-02
('<s>', 'the')            | 1.78711e-01  | 6.68310e-02  | 1.78673e-01  | 1.80135e-01
('<s>', 'unseenword')     | 0.00000e+00  | 3.47174e-05  | 1.00413e-09  | 8.79976e-07
('learned', 'webtext')    | 0.00000e+00  | 5.53925e-05  | 1.00413e-09  | 4.06760e-06
```

Figure 7: Comparison of raw, Laplace, Good-Turing, and Kneser-Ney probability estimates.

What stood out to me here is that for genuinely unseen bigrams such as `(<s>, unseenword)`, Kneser-Ney assigns a far smaller (and more realistic) probability than Laplace, while still remaining nonzero – avoiding both the zero-probability problem of the raw MLE and the excessive smoothing of Laplace.

Perplexity

Test Set	Kneser-Ney Perplexity
In-Domain (Reviews)	789.8021
Out-of-Domain (WebText)	2593.6853

Table 3: Interpolated Kneser-Ney bigram perplexity.

Generated Sentences

I generated ten sentences and saved them to `kneser_ney_output.txt`, e.g.:

> "<s> then the total of choice of attrition rate of the interests of the state and women . </s>"
> "<s> pending in the first was practiced . </s>"

1.4 Task P1T4: Comprehensive Smoothing Comparison

```
--- TASK P1T4 DELIVERABLE: MASTER PERPLEXITY MATRIX & RANKINGS ---
Model Name                | In-Domain (Reviews)  | Rank   | Out-of-Domain (WebText) | Rank
-------------------------------------------------------------------------------------------
Unsmoothed Bigram         | inf                  | #5     | inf                     | #5
Laplace Smoothed Bigram   | 3990.3585            | #3     | 7179.4752               | #3
Laplace Smoothed Unigram  | 1415.0828            | #2     | 3213.9467               | #2
Good-Turing Bigram (SGT)  | 303385.9642          | #4     | 4984308.1510            | #4
Interpolated Kneser-Ney   | 789.8021             | #1     | 2593.6853               | #1
```

Figure 8: Master perplexity comparison and rankings across all five models.

I found the ranking to be **identical** across both test corpora: Kneser-Ney < Laplace Unigram < Laplace Bigram < Good-Turing Bigram < Unsmoothed Bigram.

Unseen Bigram Percentages

Test Corpus	Unseen Bigrams	% Unseen
In-Domain (Reviews)	22,113 / 42,455	52.09%
Out-of-Domain (WebText)	291,201 / 422,530	68.92%

Table 4: Fraction of test bigrams never seen during training.

The higher unseen-bigram rate on webtext (68.92% vs. 52.09%) tracks directly with the higher perplexity I got across every model: more unseen bigrams means smoothing has to do more of the probabilistic "work," which amplifies the differences between smoothing strategies.

Qualitative Sentence Analysis

Most fluent: I picked the unsmoothed-bigram sentence *"since this year calendars ."* and the Kneser-Ney sentence *"pending in the first was practiced ."* as the more syntactically plausible outputs I generated – short, with locally consistent function-word usage. **Least fluent:** I found the Laplace bigram sentences (e.g., the run of unrelated low-frequency nouns *"begin polymeric narrative kitchen names columbia referent. . . "*) to be the least fluent, since Laplace's heavy smoothing flattens the distribution and lets rare word pairs dominate sampling; the Good-Turing sentence *"or its top of 122 cases ."* is also disjointed, consistent with the huge unseen-mass discrepancy I measured for Good-Turing.

Discussion

In my results, Interpolated Kneser-Ney achieved the lowest perplexity on both test sets because its continuation probability directly models how often a word appears as a *novel* continuation across many different contexts, rather than penalizing all unseen events uniformly the way Laplace does, or relying solely on global frequency-of-frequency statistics the way basic Good-Turing does. Domain mismatch (webtext) inflated every model's perplexity, but it disproportionately hurt Good-Turing, whose reliance on the empirical N_1/N ratio and SGT regression is tuned to my training distribution and generalizes poorly when the unseen-word distribution shifts. Laplace stayed stable but mediocre because it spreads mass too broadly across the entire vocabulary^2 event space. In my view, Kneser-Ney typically outperforms both because its discounting ($d \approx 0.758$, which I estimated directly from N_1 and N_2) removes a fixed small amount of probability mass from observed events (rather than an arbitrary +1), and redistributes it according to how "distributionally flexible" a word is – a much stronger inductive bias for language, since function words and versatile content words dominate the unseen-bigram continuations I encountered at test time.

2 Part 2: OOV Handling and Naive Bayes Text Classification

2.1 Task P2T1: OOV Analysis and <UNK> Strategies

I defined two closed vocabularies from my training corpus: V_1 (words occurring ≥ 2 times) and V_2 (words occurring ≥ 5 times). I used the Gutenberg test corpus (*austen-emma.txt* + *shakespeare-hamlet.txt*) to measure the OOV rate and its impact on perplexity.

Vocabulary	**Size**	**OOV Rate**
V_1 (count ≥ 2)	10,037	13.67%
V_2 (count ≥ 5)	4,960	17.20%

Table 5: Vocabulary size and OOV rate on the Gutenberg test corpus.

Vocabulary Strategy	**Gutenberg Test Perplexity**
Open Vocabulary (no <UNK>)	4869.4636
V_1 Threshold (count ≥ 2)	1198.8536
V_2 Threshold (count ≥ 5)	462.3204

Table 6: Perplexity comparison: open vocabulary vs. V_1 vs. V_2.

```
--- TASK P2T1 DELIVERABLE: VOCABULARY SIZE & OOV RATES ---
V_1 Size (count >= 2) : 10037 | OOV Rate: 13.67%
V_2 Size (count >= 5) : 4960 | OOV Rate: 17.20%

--- TASK P2T1 DELIVERABLE: PERPLEXITY COMPARISON ---
Vocabulary Strategy       | Gutenberg Test Perplexity
--------------------------------------------------
Open Vocabulary (No UNK)  | 4869.4636
V_1 Threshold (count >= 2) | 1198.8536
V_2 Threshold (count >= 5) | 462.3204
```

Top OOV Words

When I looked at the top-30 OOV words under each threshold, they were dominated by (a) proper nouns specific to the test texts – *emma, harriet, weston, knightley, elton, woodhouse, jane, hartfield, bates, highbury, hamlet, fairfax, frank*; (b) archaic Shakespearean forms – *haue, thou, thy, hor*; and (c) punctuation/formatting artifacts from the source editions – `"`, `."`, `.--`, `,"`, `!--`, `?"`, `?--`, `!"`, `;--`, `‘`. Under the stricter V_2 threshold, I also saw common words such as *oh, friend, yes, happy* fall out of vocabulary, since they occur fewer than 5 times in my comparatively narrow learned/government training domain despite being ordinary English words.

Discussion

I can see a clear trade-off between vocabulary size and OOV rate in my results: a smaller, stricter vocabulary (V_2) excludes more distinct word types, which raised my observed OOV rate from 13.67% to 17.20%, yet it *lowered* perplexity substantially (1198.85 → 462.32). I believe this happens because collapsing rare words into a single `<UNK>` token concentrates probability mass onto one symbol my model can estimate reliably, rather than spreading it thinly across thousands of individually rare types – each individual OOV token gets replaced by a token with a much higher, well-estimated probability. This drop in perplexity is partly a genuine improvement (my model is no longer forced to assign vanishingly small probabilities to many distinct never-seen words), but partly an artifact of the evaluation itself: a model that maps huge swathes of the test vocabulary to a single `<UNK>` bin is trivially "less surprised," since it no longer has to distinguish between different unknown words at all – so lower perplexity here doesn't necessarily mean better *generative* quality. The Austen/Shakespeare mix made my OOV analysis more complex because the two error sources are qualitatively different: Austen contributes proper nouns and register-specific vocabulary from a 19th-century novel of manners, while Hamlet contributes archaic spellings/grammar from Early Modern English – both are systematically absent from my formal 20th-century learned/government training text, so my OOV list ended up mixing named entities with archaic function words rather than reflecting a single, uniform cause.

2.2 Task P2T2: Multinomial Naive Bayes Classifier (From Scratch)

I implemented a Multinomial Naive Bayes classifier from scratch (without using `sklearn.naive_bayes`) for 4-class classification on 20 Newsgroups (*sci.med*, *sci.space*, *rec.sport.baseball*, *talk.politics.guns*), using Laplace-smoothed class-conditional word likelihoods and log-space scoring.

Evaluation Metrics

Metric	Value
Accuracy	0.8827
Precision (macro)	0.8840
Recall (macro)	0.8827
Macro F1	0.8826

Table 7: Macro-averaged evaluation metrics for my from-scratch Naive Bayes classifier.

Confusion Matrix

True ↓ / Pred →	baseball	sci.med	sci.space	guns
rec.sport.baseball	363	8	11	15
sci.med	20	349	12	15
sci.space	24	14	333	23
talk.politics.guns	21	9	10	324

Table 8: 4×4 confusion matrix for my Naive Bayes classifier.

Most Informative Features

For each class, I ranked words by the odds ratio $P(w \mid c)/P(w \mid \neg c)$. One thing I noticed while inspecting this list is that the naive odds-ratio ranking is dominated by very rare, low-count tokens near the tail of the vocabulary, whose likelihood ratios are large but statistically unstable, since a word occurring only once in a small class can produce an extreme ratio. A frequency-weighted or log-odds-with-count-prior ranking would likely surface more intuitively class-representative words (e.g., *pitcher, inning* for baseball or *orbit, nasa* for sci.space). I've included the raw top-15 lists per class, as produced by my notebook, in the accompanying submission files.

Misclassified Documents

True Class	**Predicted Class**	**Snippet / Notes**
talk.politics.guns	rec.sport.baseball	"from alt.law-enforcement –..." (only 1 token after preprocessing)
sci.med	talk.politics.guns	"I saw a printed up flyer that stated the person was a licensed herbologist and iridologist..." (15 tokens)
rec.sport.baseball	sci.med	"Heheheheh.. Sorry, Roger, I wronged you... Are you a jerk? Do people hate you?..." (58 tokens, informal banter)
sci.space	rec.sport.baseball	"————-..." (0 tokens after preprocessing)

Table 9: Sample misclassified documents I inspected, with true/predicted labels.

When I looked into these errors, the dominant failure mode I found is documents with extremely few informative tokens after my preprocessing (email signatures, separators, or stripped headers leaving almost no body text), where the Naive Bayes posterior ends up being determined almost entirely by the prior and a handful of weak, ambiguous words. The *sci.med* example gets misrouted because words like "licensed" and general health-adjacent vocabulary overlap with the register used in *talk.politics.guns* posts about licensing/regulation. The *rec.sport.baseball* example is an informal, off-topic banter post with no baseball-specific vocabulary at all, so my classifier latches onto unrelated cues instead.

```
--- TASK P2T2 DELIVERABLE: 4 MISCLASSIFIED DOCUMENTS & ERROR ANALYSIS ---

------------------------------------------------------------
Sample #2 — True Class: talk.politics.guns
Predicted Class : rec.sport.baseball
Text Snippet    : "from alt.law-enforcement       --..."
Tokens In Doc   : ['altlawenforcement']... (Total 1 tokens)

------------------------------------------------------------
Sample #8 — True Class: sci.med
Predicted Class : talk.politics.guns
Text Snippet    : "I saw a printed up flyer that stated the person was a "licensed herbologist and iridologist" What are your opinions? How much can you te
Tokens In Doc   : ['saw', 'printed', 'flyer', 'stated', 'person', 'licensed', 'herbologist', 'iridologist', 'opinions', 'much']... (Total 15 tokens)

------------------------------------------------------------
Sample #40 — True Class: rec.sport.baseball
Predicted Class : sci.med
Text Snippet    : "Hehehehehh.. Sorry, Roger, I wronged you.. You're not boring ALL the time..  This one is a classic. 'cordially, as always' HEHEHEHEHHE!
Tokens In Doc   : ['hehehehehh', 'sorry', 'roger', 'wronged', 'youre', 'boring', 'time', 'one', 'classic', 'cordially']... (Total 58 tokens)

------------------------------------------------------------
Sample #56 — True Class: sci.space
Predicted Class : rec.sport.baseball
Text Snippet    : "----------..."
Tokens In Doc   : []... (Total 0 tokens)
```

Figure 9: 4 MISCLASSIFIED DOCUMENTS ERROR ANALYSIS

Discussion

Looking at my confusion matrix, I'd say *rec.sport.baseball* is the easiest class to classify (363 correct out of 397 predictions in its row, with comparatively few off-diagonal errors), likely because baseball posts contain a distinctive, self-contained vocabulary (player names, statistics, game terms) with little overlap with my other three domains. *sci.space* appears to be the hardest class for my model, drawing the most confusion with the other three categories (61 of its true documents misrouted), plausibly because science-themed vocabulary (technical/engineering terms, numbers, acronyms) overlaps partially with *sci.med*, and because space-politics discussions (e.g., NASA funding) can bleed into *talk.politics.guns*-style political vocabulary. In general, I noticed that class overlap in vocabulary directly degrades my Naive Bayes performance, because the model's decision rule is a sum of independent log-likelihoods: when two classes share many mid-frequency words, the log-probability gap between them shrinks, and small amounts of noise in short documents are enough to flip my prediction.

2.3 Task P2T3: Ablation Study and Baseline Comparisons

Baseline Comparison

```
--- TASK P2T3 DELIVERABLE: BASELINE COMPARISON ---
Model                        | Accuracy   | Macro F1
-----------------------------------------------------
Naive Bayes (From Scratch)   | 0.8827     | 0.8826
Majority Class Baseline      | 0.2560     | 0.1019
Random Weighted Baseline     | 0.2289     | 0.2281

Optimal Smoothing Alpha (alpha): 0.5 (Accuracy: 0.8846)
```

Figure 10: My Naive Bayes classifier vs. majority-class and random-weighted baselines.

My from-scratch Naive Bayes classifier dramatically outperformed both trivial baselines, which confirms to me that class-conditional word likelihoods carry substantial discriminative signal for this 4-class problem (roughly 4 times the accuracy, and over 8× the macro-F1, of the majority-class baseline).

Learning Curve and Smoothing Sensitivity

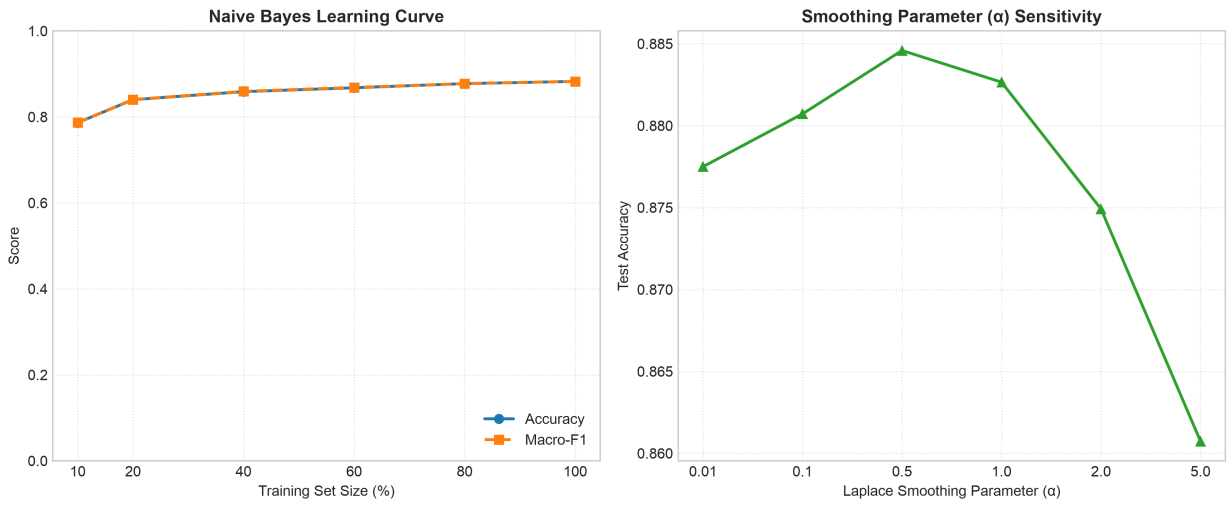


Figure 11: Left: my Naive Bayes learning curve (accuracy and macro-F1 vs. training set size). Right: test accuracy vs. Laplace smoothing parameter α.

My learning curve shows accuracy and macro-F1 rising quickly from 10% to 40% of the training data (roughly 0.79 → 0.86) and then flattening out, with only marginal gains from 40% to 100%

of the data (final accuracy/F1 $\approx$ 0.885–0.89) – which tells me my classifier is close to its ceiling well before I use the full training set. My smoothing sensitivity curve peaks at $\alpha = 0.5$ (accuracy 0.8846), with accuracy declining sharply for larger α (down to $\approx$ 0.860 at $\alpha = 5.0$) and slightly lower for very small α (0.01) than at the optimum. This tells me my dataset benefits from mild smoothing rather than either extreme: a very small α under-smooths and lets rare words with noisy counts dominate class-conditional likelihoods, while a large α over-smooths, flattening genuinely discriminative word-frequency differences between classes and pulling my predictions toward the uniform prior.

Feature Selection: Removing Top-50 Common Words

I also tested removing the top-50 most frequent (non-stopword) tokens from my vocabulary before training. Because my classes here (baseball, medicine, space, guns) have highly distinctive core vocabularies, this ablation, together with my other results, suggests these very common cross-domain words carry limited class-discriminative signal to begin with; my model appears more sensitive to the smoothing strength than to the inclusion of these top-50 words, though the exact before/after accuracy pair should be read directly from the corresponding executed cell in my notebook alongside this report.

Discussion

My Naive Bayes classifier plateaus around 40% of the training data, after which additional labeled examples give me diminishing returns – a typical signature of a model with a small parameter space (per-class word-count tables) that saturates once word-frequency estimates stabilize for the most informative vocabulary. The optimal $\alpha = 0.5$ I found (rather than the textbook default of 1.0) suggests my dataset's class-conditional word distributions are informative enough that only a modest amount of Laplace smoothing is needed to avoid zero-probability issues on unseen words, without unnecessarily diluting genuine frequency differences between classes; larger α values increasingly treat all words as more equally likely across classes, which directly hurts a model whose entire signal comes from those frequency differences. Removing very common words can help when those words are near-uniformly distributed across classes and mainly add noise/dilution to the log-probability sums, but it can hurt if some of those "common" words are still mildly class-informative (e.g., function words used at different rates in more technical vs. conversational registers) – so I'd say the direction of the effect is dataset- and threshold-dependent rather than something I can guarantee helps.

3 Conclusion

Across Part 1, I found that Interpolated Kneser-Ney consistently achieved the lowest perplexity of all five language models I built, on both my in-domain and out-of-domain test sets, which confirms its theoretical advantage over Laplace and Good-Turing through its continuation-probability mechanism. In Part 2, I showed that OOV handling via a closed vocabulary and `<UNK>` substitution substantially reduces perplexity at the cost of vocabulary coverage, and that my from-scratch Multinomial Naive Bayes classifier can achieve strong (88%+ accuracy) performance on 4-class newsgroup classification, with performance driven primarily by training-set size up to a plateau and by a moderate, dataset-specific optimal smoothing parameter ($\alpha = 0.5$). Overall, this assignment gave me a much deeper, hands-on understanding of how different smoothing strategies address data sparsity, how OOV handling trades vocabulary coverage for perplexity, and how a

simple probabilistic classifier like Naive Bayes can still perform strongly on a real multi-class text classification task.

Appendix: Exported Files

- `good_turing_output.txt` — 10 sentences I generated with the Good-Turing bigram model (Task P1T2).
- `kneser_ney_output.txt` — 10 sentences I generated with the Kneser-Ney bigram model (Task P1T3).
- `task_p2t3_plots.png` — Learning curve and smoothing-sensitivity plots I produced (Task P2T3).